

The background of the cover is a photograph of three people (two men and one woman) sitting around a table, looking at documents and a tablet. The image is overlaid with various geometric shapes: a large orange hexagon in the upper left, a white circle in the center, and several other hexagons and circles in shades of gray, blue, and brown. Some of these shapes have diagonal line patterns.

SEGURANÇA DE SISTEMAS DA INFORMAÇÃO

Fabício Felipe
Meleto Barboza

Exploração de falha de segurança

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Identificar as vulnerabilidades mais comuns.
- Analisar as vulnerabilidades.
- Aplicar o gerenciamento de vulnerabilidades.

Introdução

Quando se trabalha com tecnologia, vulnerabilidades podem surgir do dia para a noite por meio de *updates* ou de brechas de segurança descobertas. Assim, administradores de sistemas e especialistas em segurança da informação devem trabalhar em conjunto, a fim identificar e diminuir essas falhas, para que os sistemas funcionem de forma plena e sem erros ou dados incorretos em seu meio.

No capítulo, você vai ver mais sobre identificação, análise e gerenciamento de vulnerabilidades e testes de invasão.

Vulnerabilidades mais comuns

Vulnerabilidades ocorrem tanto no mundo real como no mundo virtual. Vulnerabilidades no mundo virtual são as brechas que atacantes se aproveitam para burlar o funcionamento correto do sistema ou, ainda, roubar informações deste.

Assim, a exploração de vulnerabilidades ocorre de forma rápida e sorrateira. Administradores de sistemas e especialistas em segurança da informação devem trabalhar em conjunto para identificar e diminuir essas falhas para que os sistemas funcionem de forma plena e sem erros ou dados incorretos em seu meio.

Para McClure, Scambray e Kurtz (2014), o perfil é importante para a segurança, sendo “[...] uma combinação de ferramentas e técnicas, acompanhadas

de uma boa dose de paciência e ordenação mental, os invasores podem pegar uma entidade desconhecida e reduzi-la a uma gama específica de nomes de domínio, blocos de rede, sub-redes, roteadores e endereços IP individuais de sistemas diretamente conectados à Internet, assim como muitos outros detalhes pertinentes”.

Como ponto de partida no estudo das vulnerabilidades mais comuns, tem-se um artigo publicado por Wade (2015) que cita as mais populares, sendo a ordem decrescente de ocorrências:

- qualidade do código;
- criptografia;
- vazamento de informações;
- *CRLF injection*;
- *cross-site scripting*;
- acesso indevido;
- validação de dados deficiente;
- *SQL injection*;
- gerenciamento de credenciais falho;
- Erros de *time* ou *state*;

A seguir, estudaremos o que significa cada um dos itens elencados no *ranking* caracterizados por Wade (2015).

Qualidade do código

Como principal vulnerabilidade elencada, está a criação de código ruim por parte do time de desenvolvimento responsável pelo sistema em foco. Muitas empresas investem em *frameworks* para assegurar o aceitável de qualidade nos sistemas criados por meio do emprego de boas práticas, bem como adotar medidas mais rigorosas ao enviar um novo código para o ambiente de produção.

Criptografia

Segunda colocada no *ranking*, a criptografia deve ser utilizada sempre nas comunicações via rede de computadores, garantindo que a mensagem chegue íntegra e sem ter a chance de que, caso caía em mãos erradas, essa pessoa possa ter acesso ao seu conteúdo. Com a popularização da internet e dos sistemas, a área de criptografia não deve ser negligenciada em nenhuma hipótese.

Vazamento de informações

Medalha de bronze no *ranking*, o vazamento de informações pode ocorrer em duas frentes distintas: interna ou externa. O vazamento interno seria, por exemplo, algum funcionário que verifica informações as quais não deveria ter acesso. Já o vazamento de informações externo é quando algum invasor consegue acesso ao sistema e rouba os dados contidos nele.

CRLF injection

O *CRLF injection* é um dos mais fáceis de se evitar e está na quarta posição do *ranking*. Por meio do *CRLF injection*, é possível que, pelos códigos maliciosos em lugares corretos, o invasor consiga dados confidenciais do sistema ou até mesmo do próprio computador infectado do usuário vítima.

Cross-site scripting

Também conhecido como XSS, o *cross-site scripting* utiliza *sites* com conteúdo não estático. Por meio da execução de código malicioso pelo invasor, este consegue o controle do computador da vítima e rouba sessões legítimas do usuário. Para realizar esse tipo de ataque, são usados caracteres especiais que o sistema interpreta como parte do código do próprio sistema de forma equivocada.

Acesso indevido

Outro tipo de invasão que é facilmente evitado por meio de uma consultoria adequada. O acesso indevido é quando o invasor, por meio de brechas de segurança na comunicação entre o servidor e o navegador do cliente, rouba acesso do usuário vítima e também as informações contidas na sessão.

Validação de dados deficiente

Todo campo de dados que o sistema receberá deve ser tratado. Entenda por entrada qualquer campo em que o usuário irá entrar com um valor e esse valor deve ser processado ou salvo pelo sistema. Assim, campos como nome, *login*, senha ou endereço, por exemplo, no cadastro de cliente, devem ser validados para conferir se está recebendo somente caracteres permitidos e não aceitar

entradas inválidas ou caracteres especiais, tais como exclamação, interrogação, cifrão, asterisco, etc.

SQL injection

O *SQL injection* já foi um problema maior, mas ainda continua no *ranking top 10*. Por meio de inserção de SQL, o banco de dados do sistema estará muito comprometido. Roubo de dados nesse problema é só o começo. Dados podem ser modificados e o sistema pode não corresponder mais à realidade que deveria, portanto, é um estrago dos piores.

Gerenciamento de credenciais falho

Quem, quando e o que cada usuário pode acessar. Uma manutenção nas credenciais de acesso nunca deve ser postergada, como a exclusão de permissão ou usuários, por exemplo. O sistema deve conter somente e pelo tempo necessário os usuários com permissões de acesso na alçada necessária.

Erros de *time* ou *state*

Por fim, com um ambiente complexo e recheado de atividades em paralelo, um sistema deve sempre ter a sua disposição pessoas autorizadas a desempenhar cada um dos papéis necessários ao seu bom funcionamento. Assim, evita-se a execução de códigos não autorizados por pessoas que, mesmo com as mais boas e inocentes intenções, podem prejudicar o sistema como um todo ou em parte.



Saiba mais

Das vulnerabilidades estudadas, as mais fáceis de evitar e já eliminar vários possíveis problemas futuros são: *SQL injection*, *cross-site scripting* (também conhecido por XSS) e gerenciamento de credenciais falho.

Análise de vulnerabilidades

Identificada uma vulnerabilidade, o que deve ser feito a seguir? Ao se deparar com uma vulnerabilidade no sistema ou na rede de computadores, é importante realizar os levantamentos do impacto de que essa vulnerabilidade pode ocasionar aos usuários ou dados do sistema. Com isso mensurado, deve-se ponderar sobre a correção imediata ou postergar a atividade caso o impacto não seja significativo.

A análise de vulnerabilidade é de suma importância para o desenvolvimento de correções e melhorias futuras no *software* ou na rede de computadores projetada.

A análise de vulnerabilidade é sempre importante para definir como e quando deverá ser sanada uma falha ou brecha de segurança sistêmica: imediatamente, em pequeno, médio ou longo prazo.

O início das vulnerabilidades é sustentado por um dos três pilares abaixo:

- Erros de programação: código-fonte com brechas de segurança, portanto, violáveis aos ataques.
- Falha humana: ações realizadas por pessoas consideradas vítimas de algum outro crime virtual, como links falsos, *malwares*, vírus, etc.
- Má configuração: contar com sistemas de bloqueios com configuração deficitária, portanto, não abrange a totalidade do sistema.

Para definir a correção ou não do problema, deve-se analisar a vulnerabilidade em algumas medidas para tomar a decisão sobre o que fazer. Por exemplo: se uma vulnerabilidade é muito complexa de ser resolvida e o risco de ficar vulnerável é razoável para a empresa, essa vulnerabilidade possivelmente não será solucionada até que exista tempo hábil ou necessidade para tal.

A análise de vulnerabilidade vem para suprir essa lacuna, sendo que Santos (2018) diz que ela “[...] trata basicamente do processo de identificação de falhas e vulnerabilidades (brechas) conhecidas presentes no ambiente e que o expõem a ameaças. Essas falhas podem ser causadas por diversos motivos”.

Santos (2018) também indica os objetivos da análise de vulnerabilidade da seguinte forma:

- Identificar e tratar falhas de *softwares* que possam comprometer seu desempenho, sua funcionalidade e sua segurança.

- Providenciar uma nova solução de segurança, como o uso de um bom antivírus, com possibilidade de *update* constante e implementação de sistemas de detecção e prevenção de intrusão.
- Alterar as configurações de *softwares* a fim de torná-los mais eficientes e menos suscetíveis a ataques.
- Utilizar mecanismos para bloquear ataques automatizados (*worms*, *bots*, entre outros).
- Implementar a melhoria constante do controle de segurança.
- Documentar os níveis de segurança atingidos para fins de auditoria e *compliance* com leis, regulamentações e políticas.



Fique atento

Lembre-se que o trabalho para corrigir uma vulnerabilidade deve ser mensurado sempre em uma balança, incluindo aqui os riscos envolvidos.

Aplicando o gerenciamento de vulnerabilidades

Para identificar e até mesmo mensurar uma vulnerabilidade, podemos utilizar diversas ferramentas para auxílio. Verificaremos quatro ferramentas ao todo neste capítulo, sendo:

- *Port scanner*;
- *Protocol analyzer*;
- *Vulnerability scanner*;
- *Honeypots*.

Port scanner

Com utilitários de escaneamento de portas, é possível descobrir portas que estejam abertas para acesso em servidores ou computadores da rede.

O *port scanner* mais conhecido é o *Nmap*. Com sintaxe simples e rápida, é possível descobrir em segundos quais portas estão liberadas para conexão. A seguir, veja na Figura 1 uma imagem do resultado do *Nmap*.


```
Starting Nmap 7.60 ( https://nmap.org ) at 2018-10-16 21:26 -03
Nmap scan report for ec2-34-203-199-183.compute-1.amazonaws.com (34.203.199.183)
Host is up (0.21s latency).
Not shown: 997 filtered ports
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
443/tcp   open  https
```

Figura 1. Nmap.

Audiodescrição

A imagem é uma captura de tela em formato retangular contendo um terminal com fundo escuro e texto claro. O texto é o seguinte:

Starting Nmap 7.60 (https://nmap.org) at 2018-10-16 21:26 -03

**Nmap scan report for ec2-34-203-199-183.compute-1.amazonaws.com
(34.203.199.183)**

Host is up (0.21s latency).

Not shown: 997 filtered ports

Depois há um bloco com três colunas, na ordem da esquerda para a direita: **PORT**, **STATE**, **SERVICE**.

Abaixo desse cabeçalho, aparecem três linhas:

22/tcp open ssh

80/tcp open http

443/tcp open https

Figura 1. Nmap.

Fim da audiodescrição

Como pode-se observar nas últimas três linhas do retorno do comando, existem três portas abertas nesse alvo colocado, sendo 22 com o serviço ssh, 80 com o serviço web/http e a 443 com o serviço web/https.

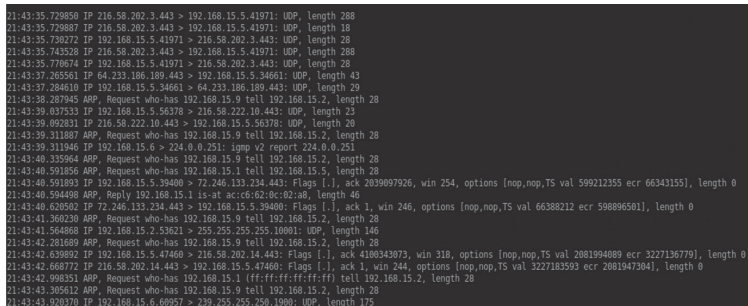
Como o alvo se trata de um *site* bem conhecido, a porta 22 estaria liberada de forma desnecessária. Ninguém deveria acessar esse servidor por ssh diretamente, mas, sim, por meio de outro *host* com vpn e outros degraus de segurança.

Protocol analyzer

Este grupo contempla as ferramentas de análise de tráfego de rede, podendo verificar com base em protocolo trafegado (udp ou tcp), endereço IP de origem, endereço IP de destino, entre outras informações do pacote de dados

em trânsito.

Nesta área de *protocol analyzer*, a ferramenta chamada *tcpdump* é a mais difundida. Isso ocorre por ela ser extremamente leve, com sintaxe fácil de entender, precisa e muito completa em suas opções. Veja a Figura 2.



```

21:43:35.729850 IP 216.58.202.3.443 > 192.168.15.5.41971: UDP, length 288
21:43:35.729887 IP 216.58.202.3.443 > 192.168.15.5.41971: UDP, length 18
21:43:35.730272 IP 192.168.15.5.41971 > 216.58.202.3.443: UDP, length 28
21:43:35.735328 IP 216.58.202.3.443 > 192.168.15.5.41971: UDP, length 288
21:43:35.776074 IP 192.168.15.5.41971 > 216.58.202.3.443: UDP, length 28
21:43:37.265561 IP 64.233.186.189.443 > 192.168.15.5.34661: UDP, length 43
21:43:37.284610 IP 192.168.15.5.34661 > 64.233.186.189.443: UDP, length 29
21:43:38.287945 ARP, Request who-has 192.168.15.9 tell 192.168.15.2, length 28
21:43:39.037533 IP 192.168.15.5.56378 > 216.58.222.10.443: UDP, length 23
21:43:39.092831 IP 216.58.222.10.443 > 192.168.15.5.56378: UDP, length 20
21:43:39.311887 ARP, Request who-has 192.168.15.9 tell 192.168.15.2, length 28
21:43:39.311946 IP 192.168.15.6 > 224.0.0.251: igmp v2 report 224.0.0.251
21:43:40.335964 ARP, Request who-has 192.168.15.9 tell 192.168.15.2, length 28
21:43:40.591856 ARP, Request who-has 192.168.15.1 tell 192.168.15.5, length 28
21:43:40.591893 IP 192.168.15.5.39400 > 72.246.133.234.443: Flags [I], ack 2039097926, win 254, options [nop,nop,TS val 599212355 ecr 66343155], length 0
21:43:40.594498 ARP, Reply 192.168.15.1 is-at ac:c6:62:0c:02:a8, length 46
21:43:40.626592 IP 72.246.133.234.443 > 192.168.15.5.39400: Flags [I], ack 1, win 246, options [nop,nop,TS val 66388212 ecr 598896581], length 0
21:43:41.366230 ARP, Request who-has 192.168.15.9 tell 192.168.15.2, length 28
21:43:41.564868 IP 192.168.15.2.53621 > 255.255.255.255.10801: UDP, length 146
21:43:42.281889 ARP, Request who-has 192.168.15.9 tell 192.168.15.2, length 28
21:43:42.439892 IP 192.168.15.5.47486 > 216.58.202.14.443: Flags [I], ack 480843073, win 318, options [nop,nop,TS val 2081994089 ecr 3227136779], length 0
21:43:42.668772 IP 216.58.202.14.443 > 192.168.15.5.47486: Flags [I], ack 1, win 244, options [nop,nop,TS val 3227163593 ecr 2081947304], length 0
21:43:42.998351 ARP, Request who-has 192.168.15.1 (ff:ff:ff:ff:ff:ff) tell 192.168.15.2, length 28
21:43:43.305612 ARP, Request who-has 192.168.15.9 tell 192.168.15.2, length 28
21:43:43.928770 IP 192.168.15.6.69921 > 220.235.255.256.1000: UDP, length 175

```

Figura 2. *Tcpdump*.

Audiodescrição

A imagem é uma captura de tela retangular contendo um terminal com fundo escuro e diversas linhas de texto claras exibindo registros de tráfego de rede. Cada linha apresenta horário, protocolo e informações de origem, destino e tamanho dos pacotes. A leitura segue de cima para baixo. O conteúdo exibido é:

```

21:43:35.729850 IP 216.58.202.3.443 > 192.168.15.5.41971: UDP, length 288
21:43:35.729887 IP 216.58.202.3.443 > 192.168.15.5.41971: UDP, length 18
21:43:35.730272 IP 192.168.15.5.41971 > 216.58.202.3.443: UDP, length 28
21:43:35.735328 IP 216.58.202.3.443 > 192.168.15.5.41971: UDP, length 288
21:43:35.776074 IP 192.168.15.5.41971 > 216.58.202.3.443: UDP, length 28
21:43:37.265561 IP 64.233.186.189.443 > 192.168.15.5.34661: UDP, length 43
21:43:37.284610 IP 192.168.15.5.34661 > 64.233.186.189.443: UDP, length 29
21:43:38.287945 ARP, Request who-has 192.168.15.9 tell 192.168.15.2, length 28
21:43:39.037533 IP 192.168.15.5.56378 > 216.58.222.10.443: UDP, length 23
21:43:39.092831 IP 216.58.222.10.443 > 192.168.15.5.56378: UDP, length 20
21:43:39.311887 ARP, Request who-has 192.168.15.9 tell 192.168.15.2, length 28
21:43:39.311946 IP 192.168.15.6 > 224.0.0.251: igmp v2 report 224.0.0.251
21:43:40.335964 ARP, Request who-has 192.168.15.9 tell 192.168.15.2, length 28
21:43:40.591856 ARP, Request who-has 192.168.15.1 tell 192.168.15.5, length 28
21:43:40.591893 IP 192.168.15.5.39400 > 72.246.133.234.443: Flags [I], ack
2039097926, win 254, options [nop,nop,TS val 599212355 ecr 66343155], length 0
21:43:40.594498 ARP, Reply 192.168.15.1 is-at ac:c6:62:0c:02:a8, length 46

```

```
21:43:40.620502 IP 72.246.133.234.443 > 192.168.15.5.39400: Flags [.], ack 1, win
246, options [nop,nop,TS val 66388212 ecr 598896501], length 0
21:43:41.360230 ARP, Request who-has 192.168.15.9 tell 192.168.15.2, length 28
21:43:41.564868 IP 192.168.15.2.53621 > 255.255.255.255.10001: UDP, length 146
21:43:42.281689 ARP, Request who-has 192.168.15.9 tell 192.168.15.2, length 28
21:43:42.639892 IP 192.168.15.5.47460 > 216.58.202.14.443: Flags [.], ack
4100343073, win 318, options [nop,nop,TS val 2081994089 ecr 3227136779], length 0
21:43:42.709980 IP 21658.202.14.443 > 192.168.15.5.47460: Flags [.], ack 1, win 244,
options [nop,nop, TS val 2081994089 ecr 3227136779], length 0
21:43:42.840152 ARP, Request who-has 192.168.15.1 (ff:ff:ff:ff:ff:ff) tell 192.168.15.12
length 28
21:43:43.305631 ARP, Request who-has 192.168.15.9 tell 192.168.15.6, length 28
21:43:43.920370 IP 192.168.15.6.60957 > 239.255.255.250.1900: UDP, length 175
```

Figura 2. *Tcpdump*.

Fim da audiodescrição.

Na Figura 2, é possível ver o resultado de um *tcpdump* na máquina local. Repare que a ordem de apresentação do resultado é:

- Horário;
- IP de origem;
- porta de origem;
- IP de destino;
- porta de destino;
- protocolo;
- tamanho.

Para lermos o resultado apresentado tomando por base as informações da primeira linha da imagem capturada, temos:

- horário: 21h43m35s
- IP de origem: 216.58.202.3
- porta de origem: 443
- IP de destino: 192.168.15.5
- porta de destino: 41971
- protocolo: UDP
- tamanho: 288

Assim, a leitura direta ficaria algo semelhante a:

às 21h43m35s, o IP origem 216.58.202.3 e porta 443 transmitiu informações/pacotes para o IP destino 192.168.15.5 na porta 41971 utilizando protocolo UDP e comprimento de 288.

Vulnerability scanner

Grupo completo de ferramentas que indicam uma varredura completa do sistema, exibindo dados de versões de *softwares*, atividades, *updates* necessários, sistema operacional, banco de dados, etc.

Honeypots

Para atrair invasores, coletar informações sobre estes e, até mesmo, distrair a sua atenção do sistema que realmente é importante, deve-se utilizar a ideia de *honeypots*, que são iscas criadas dentro da rede ou sistema para direcionar as invasões para esse ambiente e, dessa forma, livrar o ambiente.



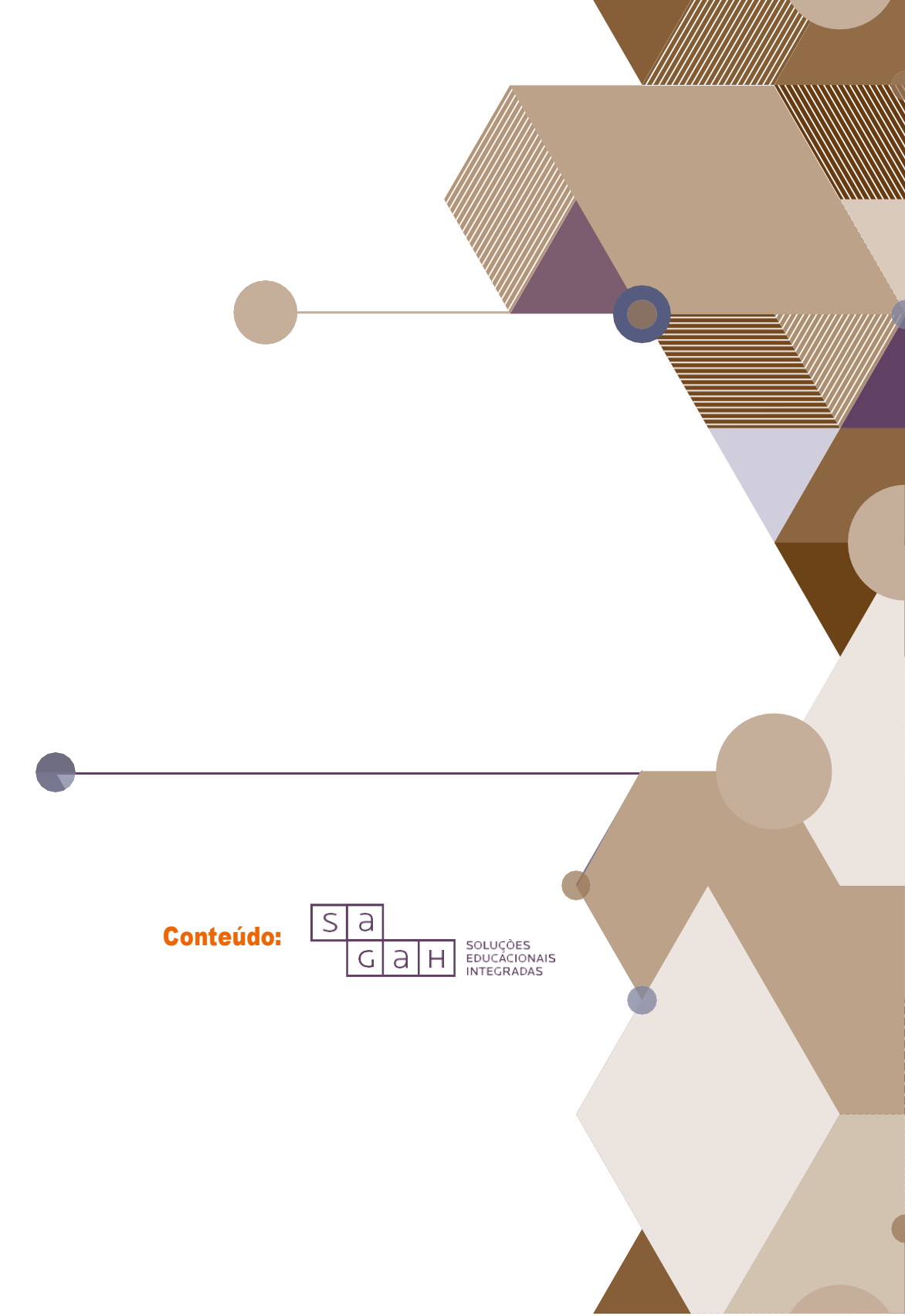
Referências

MCCLURE, S.; SCAMBRAY, J.; KURTZ, G. *Hackers expostos*: segredo e soluções para a segurança de redes. 7. ed. Porto Alegre: Bookman, 2014.

SANTOS, A. H. O. Análise de vulnerabilidades: o que é e qual a importância para a organização? *União Geek*, 27 jan. 2018. Disponível em: <<https://www.uniaoageek.com.br/analise-de-vulnerabilidades-importancia/>>. Acesso em: 23 dez. 2018.

WADE, E. 10 common security vulnerabilities. *Veracode*, 2 nov. 2015. Disponível em: <<https://www.veracode.com/blog/2015/09/10-common-security-vulnerabilities-and-markets-they-impact-sw>>. Acesso em: 23 dez. 2018.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.



Conteúdo:

S	a	
G	a	H

SOLUÇÕES
EDUCACIONAIS
INTEGRADAS